

Reviewer Pack — Möbius Coinvariant of Minterm Intersection Lattice Bounds k...

Entry 29dff0c510e9 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 08:12:59 UTC

Möbius Coinvariant of Minterm Intersection Lattice Bounds k-CLIQUE Monotone DNF

Entry ID: 29dff0c510e9 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 08:12:59 UTC

1. Conjecture

Field A (mathematical branch): Rota-Whitney Möbius function theory of intersection-closure lattices (the alternating-zeta-inversion calculus of finite meet-semilattices, classically used in subspace-arrangement combinatorics and almost never applied to monotone DNF analysis)

Field B (complexity object): Monotone DNF size for the k-CLIQUE indicator (Razborov 1985 sunflower regime; the gateway to feasible-interpolation lower bounds for Extended Frege via monotone-circuit-to-EF transfer)

Statement:

For a monotone DNF F on N variables with minimal-minterm family $M(F) = \{M_1, \dots, M_m\} \subseteq 2^{[N]}$, let $L(F)$ be the meet-semilattice generated by $\{\emptyset, [N]\} \cup M(F)$ under intersection, ordered by inclusion, with Möbius function μ_L . Define $\Lambda(F) = \log_2(1 + \sum_{x \in L(F)} |\mu_L(0, x)|)$. We conjecture: (i) for every monotone DNF F of size s on $N = C(n, 2)$ graph variables, $\Lambda(F) \leq 4 \cdot \log_2(s+1) \cdot \log_2(n+1)$; (ii) for the k-CLIQUE indicator with $k=3$ on n vertices, every monotone DNF F representing it satisfies $\Lambda(F) \geq (1/8) \cdot n \cdot \log_2(n)$. A single (s, F) pair with $\Lambda(F) > 4 \cdot \log_2(s+1) \cdot \log_2(n+1)$, or a single triangle-DNF F with $\Lambda(F) < (1/8) \cdot n \cdot \log_2(n)$, refutes the conjecture.

Rationale (proposer's reasoning):

Razborov's sunflower argument shows triangle minterms have rigid pairwise-intersection patterns; Möbius mass on the intersection lattice is exactly the alternating overlap signature that random/structured small DNFs (whose intersection lattices are nearly Boolean and hence have μ -values cancelling) cannot replicate. Möbius coinvariants are subadditive under join (Whitney 1935; Rota 1964), giving a natural conjunction-submodularity hook, and their values are computable in $O(|L(F)|^2)$ time by zeta inversion — a concrete computational shortcut that avoids any reliance on uncomputable algebraic invariants.

Taxonomy category: PROOF_COMPLEXITY_EXT_FREGE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c74132490be02558

¹N

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across $n_{\text{graph}} \in \{4, 5, 6, 7\}$ with 30 random monotone DNFs per $s \in \{n, 2n, 4n, 8n\}$ (seeds 0–29): (i) every random F must satisfy $\Lambda(F) \leq 4 \cdot \log_2(s+1) \cdot \log_2(n+1)$; (ii) the canonical triangle-DNF must satisfy $\Lambda(F_{\text{tri}}) \geq (1/8) \cdot n \cdot \log_2(n)$ and exceed max random Λ on $\geq 80\%$ of $(n_{\text{graph}}, \text{seed})$ pairs.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.90	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Möbius function intersection lattice monotone DNF lower bounds - monotone circuit complexity k-clique Razborov sunflower lattice Möbius - minterm intersection semilattice monotone formula size clique

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from itertools import combinations, chain

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def powerset(s):
        return list(chain.from_iterable(combinations(s, r) for r in range(len(s)+1)))

    def mobius_function(lattice):
        n = len(lattice)
        mu = [[0] * n for _ in range(n)]
        mu[0][0] = 1
        for i in range(1, n):
            for j in range(i-1, -1, -1):
                if lattice[j].issubset(lattice[i]):
                    mu[i][j] = -mu[i-1][j]
                    break
            for j in range(i+1, n):
```

```

        if lattice[j].issubset(lattice[i]):
            mu[i][j] = mu[i-1][j]
    return mu

def zeta_inversion(mu):
    n = len(mu)
    zeta = [[0] * n for _ in range(n)]
    for i in range(n):
        zeta[0][i] = 1
        for j in range(1, i+1):
            zeta[j][i] = sum(zeta[k][i-1] * mu[i-j+k][k] for k in range(j))
    return zeta

def log2(x):
    return math.log2(x)

def triangle_dnf(n):
    edges = list(combinations(range(n), 2))
    minterms = []
    for edge_set in combinations(edges, n-3):
        minterm = set()
        for edge in edge_set:
            minterm.update(edge)
        minterms.append(minterm)
    return minterms

def random_monotone_dnf(n, s, min_minterm_size=2, max_minterm_size=5):
    minterms = []
    while len(minterms) < s:
        minterm_size = random.randint(min_minterm_size, max_minterm_size)
        minterm = set(random.sample(range(n), minterm_size))
        if all(not m.issubset(minterm) for m in minterms):
            minterms.append(minterm)
    return minterms

def compute_lambda(lattice):
    mu = mobius_function(lattice)
    zeta = zeta_inversion(mu)
    return sum(abs(zeta[i][0]) for i in range(1, len(lattice)))

n_graphs = [4, 5, 6, 7]
k = 3
max_random_lambda = 0

for n_graph in n_graphs:
    n = math.comb(n_graph, 2)
    triangle_minterms = triangle_dnf(n_graph)
    L_triangle = powerset(triangle_minterms)
    mu_triangle = mobius_function(L_triangle)
    zeta_triangle = zeta_inversion(mu_triangle)
    lambda_triangle = sum(abs(zeta_triangle[i][0]) for i in range(1, len(L_triangle)))

    if lambda_triangle > max_random_lambda:
        max_random_lambda = lambda_triangle

    for s in [n, 2*n, 4*n, 8*n]:

```

```

    random_minterms = random_monotone_dnf(n, s)
    L_random = powerset(random_minterms)
    mu_random = mobius_function(L_random)
    zeta_random = zeta_inversion(mu_random)
    lambda_random = sum(abs(zeta_random[i][0]) for i in range(1, len(L_random)))

    if lambda_random > max_random_lambda:
        max_random_lambda = lambda_random

conjecture_holds = (lambda_triangle >= (1/8) * n_graphs[0] * log2(n_graphs[0])) and \
    all(lambda_random <= 4 * log2(s+1) * log2(n+1) for s in [n, 2*n, 4*n, 8*n])

return {
    "metric_name": "Lambda",
    "metric_value": lambda_triangle,
    "instances_tested": len(n_graphs),
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else f"Triangle-DNF: Lambda(F)={lambda_triangle}",
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(30))

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_lambda = sum(r["metric_value"] for r in results) / len(results)
    std_lambda = math.sqrt(sum((r["metric_value"] - mean_lambda) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_lambda} std={std_lambda} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        print(f"RESULT: FALSIFIED counterexample=\"Lambda(F_triangle)<(1/8)*n*log2(n)\" first_fail={...}")
    else:
        print("RESULT: INCONCLUSIVE insufficient evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_872b909c.py", line 115, in <module>

result = run_trial(seed)

^^^^^^^^^^^^^^^^

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_872b909c.py", line 81, in run_trial

mu_triangle = mobius_function(L_triangle)

^^

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_872b909c.py", line 29, in mobius_function
    if lattice[j].issubset(lattice[i]):
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

AttributeError: 'tuple' object has no attribute 'issubset'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with an AttributeError ('tuple' object has no attribute 'issubset') in the Möbius function computation before any data was produced, so neither support nor falsification conditions could be evaluated. | next: Fix the lattice representation by converting elements to frozensets (or apply issubset to the underlying set) before calling mobius_function, then rerun the pre-registered sweep.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	232032
2	preregistration	claude_max	opus	0	0	7523
3	novelty	claude_max	opus	0	0	3182
4	novelty	claude_max	opus	0	0	7086
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17167
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14183
7	judge	claude_max	opus	0	0	3860

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 285033 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/29dff0c510e9.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/29dff0c510e9.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/29dff0c510e9.tar.gz` (if generated)